

5-2 · Spark 마스터-워커 구축 · 정리·회고 0.5H

정리·회고

명령 여덟 줄과 5-1·5-2 차이를 한 장으로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

- ① 클러스터 명령 여덟 줄을 「하려는 일」로 정리한다
- ② 5-1(한 대)과 5-2(클러스터)의 차이 다섯 가지를 말한다
- ③ 자주 겪는 오류 네 가지의 원인과 해결을 안다
- ④ 마스터 UI 200 · MinIO 컨테이너 — 5-3 준비 상태를 확인한다

5-3에서는 이 클러스터에서 MinIO의 Iceberg 테이블을 읽습니다

명령 요약 — 하려는 일로 찾는 여덟 줄

하려는 일	명령
클러스터 기동	<code>docker compose up -d</code>
워커 수 조절	<code>docker compose up -d --scale spark-worker=N</code>
상태 확인	<code>docker compose ps</code> · 마스터 UI 8080
잡 제출	<code>spark-submit --master spark://spark-master:7077</code> 파일
자원 제한	<code>--total-executor-cores N --executor-memory Ng</code>
파일 넣기	<code>docker compose cp</code> 파일 <code>spark-master:/tmp/</code>
로그 보기	<code>docker compose logs spark-master · spark-worker</code>
정지	<code>docker compose stop</code>

잡 제출은 `docker compose exec spark-master /opt/spark/bin/spark-submit` 으로 — 마스터 컨테이너 안에서

5-1 과 5-2 의 차이 — master 값 하나에서

	5-1 (한 대)	5-2 (클러스터)
master 값	local[*]	spark://spark-master:7077
익스큐터	드라이버가 겸함	워커 위에 따로 뜸
UI	4040만	8080(마스터) + 4040(잡)
자원 조절	없음	--executor-* 옵션
워커 추가	불가	--scale 로 즉시

코드는 그대로 — master 값 하나가 바뀌고, 그 뒤로 익스큐터·UI·자원 조절·워커 추가가 달라진다

여덟 개를 스스로 확인해 보세요

- 1 워커와 익스큐터의 차이를 설명할 수 있다
- 2 잡을 제출했을 때 벌어지는 여섯 단계를 순서대로 말할 수 있다
- 3 `docker compose up` 한 번으로 마스터-워커 클러스터가 기동된다
- 4 마스터 UI(8080)에서 워커의 **ALIVE** 상태와 코어/메모리를 확인했다
- 5 `--scale` 로 워커를 늘렸다 줄이며 UI 가 따라 바뀌는 것을 봤다
- 6 직접 작성한 PySpark 스크립트를 클러스터에서 실행했다
- 7 master 값이 `local[*]` 에서 `spark://...` 로 바뀐 것을 확인했다
- 8 자원을 과하게 요구해 **WAITING** 을 만들고, 두 잡을 동시에 돌려 봤다

증상으로 찾는 원인과 해결

증상	원인	해결
Workers 표가 비어 있다	마스터 주소 오타 · 워커가 마스터보다 먼저 뜸	spark-master:7077 확인 · restart spark-worker
port is already allocated	8080·7077 을 다른 것이 사용 중(4주차 Trino)	점유 컨테이너 내리기 · 8081:8080 으로 매핑
not accepted any resources	요청한 코어·메모리를 채울 워커가 없다	워커 등록 확인 · --executor-memory 2g 이하
--scale 뒤 컨테이너가 바로 죽음	Docker 에 할당된 메모리를 넘었다	logs 확인 · 워커 수---memory 줄이기

16GB 환경에서는 워커 1~2개가 적당 — 「기다리는 중」인 경고를 멈춤으로 오해하지 않기

자주 나오는 질문 네 가지

1

워커를 늘리면 항상 빨라지나요?

아니다 — 데이터가 작으면 나누고 모으는 비용이 더 크다. 「한 대로 되면 한 대로」

2

마스터가 죽으면 어떻게 되나요?

돌던 잡은 계속되지만 새 잡을 제출할 수 없다 — 운영에서는 마스터를 이중화(HA)

3

실무에서도 Standalone 을 쓰나요?

소규모 전용 클러스터에서는 쓴다 — 요즘 신규 구축은 Kubernetes 가 많고, 개념은 같다

4

워커마다 다른 사양이어도 되나요?

된다 — --cores 와 --memory 를 워커마다 다르게 주면 마스터가 알아서 배치

다음 차시 준비

5-3 을 위한 상태 확인 — 세 가지

1

docker compose ps

~/data-eng/week5 의 클러스터가 Up 상태

2

curl ... localhost:8080 → 200

「마스터 UI: 200」이 찍히면 살아 있다

3

docker ps -a | grep -E "minio|rest"

4주차 minio·rest 컨테이너가 정지 상태라도 남아 있으면 준비 끝

5-3 에서 할 것

Iceberg

테이블 읽기

다음 회차 예고

5-3 · Iceberg · MinIO 연결과 대용량 분석 준비

01

이론 1H

레이크하우스와
Spark 연동 구조

02

실습 1.5H

Spark ↔ MinIO
(s3a) 연결

03

실습 1H

Iceberg 카탈로그
구성·테이블 생성

04

정리 0.5H

정리·회고
산출물 확인

준비: 4주차 MinIO 컨테이너 필요 · 5-2 compose 에 MinIO 서비스를 합쳐 같은 네트워크에서 · Iceberg 스냅샷·메타데이터 복습

5-2 · 정리·회고 0.5H 마무리

다음 회차는 5-3 Iceberg · MinIO 연결

5-2 클러스터를 s3a 커넥터로 MinIO 에 연결하고 Iceberg 카탈로그를 등록해 SQL 로 레이크하우스 테이블을 만듭니다

✓ s3a 커넥터로 MinIO 를 S3 처럼 읽고 쓰기

✓ hadoop-aws · iceberg-spark-runtime 왜 필요한가

✓ SparkCatalog 등록 · SQL 로 테이블 생성

✓ MinIO 콘솔에서 metadata/data 파일 확인

준비할 것

4주차 MinIO 컨테이너 · Iceberg 복습